

55. Devant, derrière : le buisson sur Game Boy Color, poserDevant()

Les deux buissons du 110.6, mais le second posé en UNE ligne : `poserDevant(c, l, BUISSON, 2)`.
Même écran, mêmes octets, moins à écrire.

1. Ce qu'il faut comprendre

Le programme des deux buissons (le 110.6), avec un seul changement : le second buisson n'est plus posé avec deux lignes, `poser()` puis `teindre(..., 2 | DEVANT)`, mais avec **une seule**, `poserDevant()`.

Ce qui est nouveau ici : `poserDevant(colonne, ligne, tuile, palette)`, une fonction de la console. Elle fait les deux gestes d'un coup : poser la tuile, et la mettre dans sa palette **devant** les lutins. C'est un peu comme une classe en HTML : on dit « devant » en posant la case, et on n'y pense plus.

Elle est légère, et même gratuite : le compilateur la **remplace** par les deux lignes qu'on écrivait avant. La cartouche a exactement les mêmes octets qu'au 110.6. Et ensuite, pendant le jeu, rien n'est vérifié : c'est la console qui dessine la case devant le héros, toute seule, en dessinant l'écran.

Le même écran qu'à la leçon d'avant (Game Boy normale), mais autrement : là, le programme changeait le héros de côté avec un `if`, selon sa place. Ici, grâce à `DEVANT`, c'est **la case** qui le dit : le héros n'a plus rien à savoir.

Le premier buisson garde l'ancienne façon, `poser()` puis `teindre(..., 2)`, sans `DEVANT`. On compare les deux d'un coup d'œil.

À toi : pose le premier buisson avec `poserDevant()`, lui aussi. Le héros passe derrière les deux, et `#include <teindre>` ne sert plus à rien : tu peux l'enlever.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **762 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	346 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), teindre() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	128 o	← CopierTuiles — 6 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

```
"EN UNE LIGNE"      12 o ← texte() – un texte gravé
Lettres gravées : EGILNU
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
couleurFond() × 6    72 o sur place, dans main()
teindre()     × 2    48 o sur place, dans main() + AdresseCase (10 o)
couleurLutin() × 3    36 o sur place, dans main()
poser()       × 2    26 o sur place, dans main() + AdresseCase (10 o)
sprite()      × 1    23 o sur place, dans main()
ecran()       × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
teindreLutin() × 1    16 o sur place, dans main()
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()       × 1     6 o sur place, dans main() + AttendreImage (28 o)
images()      × 1     2 o sur place, dans main()
```

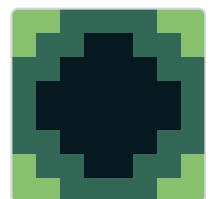
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME (Game Boy Color) : les deux buissons du 110.6. Le second est
// posé en UNE ligne avec poserDevant() ; l'écran ne change pas.
//
// Ce qui est nouveau : poserDevant(colonne, ligne, tuile, palette).
//
// AVANT (2 lignes)                                MAINTENANT (1 ligne)
// poser(c, 1, BUISSON);                            poserDevant(c, 1, BUISSON, 2);
// teindre(c, 1, 2 | DEVANT);
//
// Le compilateur remplace la ligne de droite par les deux de gauche : la
// cartouche a les mêmes octets. Rien n'est vérifié pendant le jeu : c'est la
// console qui dessine la case devant le héros, toute seule.

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <couleurFond>    // choisit une couleur d'une palette du fond
#include <couleurLutin>   // choisit une couleur d'une palette des lutins
#include <ecran>          // éteint ou rallume l'écran
#include <poser>          // pose une tuile sur une case du fond
#include <teindre>        // met une case du fond dans une palette
#include <poserDevant>    // pose une tuile qui passe devant les lutins
#include <texte>          // écrit un texte à l'écran
#include <sprite>         // place un lutin de 8 × 8 au pixel près
#include <teindreLutin>   // met un lutin dans une palette
```

2. UN DESSIN — BUISSON

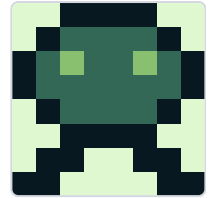
```
Tuile BUISSON = {          // Une touffe de feuilles : AUCUN 0, elle est toute pleine.
    "11222211",           // (des coins en 0 laisseraient un trou là où deux
    "12233221",           // touffes se touchent : on y verrait le héros)
    "22333322",
    "23333332",
    "23333332",
    "22333322",
    "12233221",
    "11222211",
};
```



à l'écran

3. UN DESSIN — HEROS

```
Tuile HEROS = { // 0 transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "3300033",
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    couleurFond(0, 0, 20, 28, 31); // palette 0 : le ciel et le texte
    couleurFond(0, 3, 2, 4, 12);
    couleurFond(2, 0, 20, 28, 31); // palette 2 : le ciel entre les feuilles,
    couleurFond(2, 1, 16, 30, 8); // vert clair,
    couleurFond(2, 2, 6, 22, 4); // vert,
    couleurFond(2, 3, 0, 10, 2); // vert sombre.
    couleurLutin(0, 1, 31, 28, 20); // le héros : rose clair,
    couleurLutin(0, 2, 31, 4, 2); // rouge,
    couleurLutin(0, 3, 10, 0, 0); // contour très sombre.

    ecran(0); // Dessin écran éteint.
    texte(1, 1, "EN UNE LIGNE");
    for (uint8_t l = 8; l < 10; l++) { // Lignes 8 et 9 :
        for (uint8_t c = 4; c < 6; c++) { // buisson 1, l'ancienne façon :
            poser(c, l, BUISSON);
            teindre(c, l, 2); // palette 2, sans DEVANT.
        }
        for (uint8_t c = 12; c < 14; c++) { // buisson 2, la nouvelle façon :
            poserDevant(c, l, BUISSON, 2); // posé, palette 2, DEVANT : une ligne.
        }
    }
    ecran(1); // On rallume.

    uint8_t x = 0; // La place du héros, en pixels.

    while (true) { // La boucle du jeu :
        image();

        if (images() % 2 == 0) x++; // Un pixel toutes les 2 images.
        if (x > 152) x = 0;

        sprite(0, x, 68, HEROS);
        teindreLutin(0, 0);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le même écran qu'au 110.6 : le héros passe devant le buisson de gauche, derrière celui de droite.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ buisson 1 : le héros (rouge) est devant (31/4/2)
- ✓ buisson 2, poserDevant : on voit le buisson (vert) (6/22/4)
- ✓ la case (12, 8) : palette 2 et DEVANT, comme avec teindre (130)

← [54. Devant, derrière : le buisson sur Game Boy normale, devant puis derrière](#)

[sommaire](#)

[56. Devant, derrière : le tuyau en une ligne, spriteDerriere\(\). →](#)